DEEP LEARNING / NLP · SHORT LESSON · ASIA EURO UNIVERSITY (MSC IT ENGINEERING)

Attention Mechanism & Transformer

An easy-to-follow summary of Lesson 3–5, with real-world examples, one worked calculation, and the four C5 discussion questions answered.

What you should be able to do after this lesson

- **Understand (C2):** explain Self-Attention, Query/Key/Value, Multi-Head Attention and Positional Encoding.
- **Apply (C3):** compute Scaled Dot-Product Attention by hand and in Python.
- **Analyze (C4):** read attention weights from BERT / XLM-R on Khmer sentences.
- **Evaluate (C5):** judge what attention patterns can and cannot prove.

1. Why did we move from RNN/LSTM to Attention?

Before 2017, most language models were **RNNs or LSTMs**. They read a sentence one word at a time, from left to right, and try to squeeze everything they have read into one "memory" vector.

Real-world analogy: the telephone game. Ten people stand in a line and whisper a message from one to the next. By the tenth person, the start of the message is distorted or lost. An RNN has the same problem: by the end of a long sentence, early words have faded. Attention changes the rules so that *every person can hear the original speaker directly*.

Problem with RNN/LSTM	What it means	How Attention fixes it
Information bottleneck	The whole sentence is compressed into *one* context vector.	**Dynamic context:** the decoder can look at *all* encoder hidden states.
Vanishing gradient	Information from early words is lost in long sequences.	**Weighted focus:** each word connects directly to the relevant words, however far away they are.
Sequential computation	Word 10 must wait for words 1–9, so the work can't be parallelized.	**Parallel processing:** the whole sentence is processed at once on a GPU.

Example: "*The book that my friend from Siem Reap lent me last year during the Water Festival was amazing.*" To choose "was" (not "were"), the model has to link it back to "book", which is 14 words earlier. An RNN often forgets it. Attention links the two directly.

2. Query, Key and Value (Q, K, V)

Every word (token) is turned into three vectors by multiplying its embedding **X** by three learned weight matrices:

$$Q = X \cdot W_Q \quad | \quad K = X \cdot W_K \quad | \quad V = X \cdot W_V$$

Query (Q)
"What am I looking for?" The current token asks the other tokens a question.

Key (K)
"What do I contain?" A label on each token, used to check how well it matches the query.

Value (V)
"Here is my actual content." The information that gets passed on once a match is found.

Real-world analogy: searching on YouTube. You type "*Khmer cooking amok*", which is your **Query**. YouTube compares it with every video's title and tags, which are the **Keys**. The best matches get high scores, and you then watch the actual videos, which are the **Values**. Attention does this "soft search" for every word in the sentence at the same time, and it blends the values by score instead of picking only one.

3. The core formula: Scaled Dot-Product Attention

$$\text{Attention}(Q, K, V) = \text{softmax}\left(Q \cdot K^T / \sqrt{d_k} \right) \cdot V$$

Step 1 · Similarity
$Q \cdot K^T$
How well does each word match each other word?

→

Step 2 · Scale
$\div \sqrt{d_k}$
Keep the numbers in a healthy range (variance control).

→

Step 3 · Softmax
Turn scores into weights that add up to 1.

→

Step 4 · Weighted sum
weights × V
Mix the information.

Worked example (the same numbers as the class exercise)

Inputs: 4 tokens, $d_k = 2$, so $\sqrt{d_k} \approx 1.414$.

`Q = K = [[1,0],[0,1],[1,1],[0,0]]  V = [[1,2],[3,4],[5,6],[7,8]]`

Step 1: $Q \cdot K^T$

	W1	W2	W3	W4
R1	1	0	1	0
R2	0	1	1	0
R3	1	1	2	0
R4	0	0	0	0

Step 2: divide by √2

R1 → [0.707, 0, 0.707, 0]

R3 → [0.707, 0.707, 1.414, 0]

Step 3: softmax per row (attention weights)

	W1	W2	W3	W4
R1	.335	.165	.335	.165
R2	.165	.335	.335	.165
R3	.221	.221	.449	.109
R4	.250	.250	.250	.250

Step 4: weights × V (output)

R1 = [3.660, 4.660] R2 = [4.000, 5.000]

R3 = [3.891, 4.891] R4 = [4.000, 5.000]

How to check R1 by hand: $e^{0.707} \approx 2.028$, so the row sum is 2.028 + 1 + 2.028 + 1 = 6.056, and the weights are 2.028/6.056 = 0.335 and 1/6.056 = 0.165. Output = 0.335·[1,2] + 0.165·[3,4] + 0.335·[5,6] + 0.165·[7,8] = [3.66, 4.66].

What it shows: Token 4 is all zeros, so it matches nothing and spreads its attention equally (0.25 each). Token 3 = [1,1] matches itself best (0.449).

The same thing in Python

```
import numpy as np
def softmax(x, axis=-1):
    e = np.exp(x - x.max(axis=axis, keepdims=True))   # subtract max for numerical stability
    return e / e.sum(axis=axis, keepdims=True)

def scaled_dot_product_attention(Q, K, V, mask=None):
    d_k = Q.shape[-1]
    scores = Q @ K.T / np.sqrt(d_k)
    if mask is not None:                    # causal mask: the decoder can't see future words
        scores = np.where(mask == 0, -1e9, scores)
    w = softmax(scores)
    return w @ V, w

causal_mask = np.tril(np.ones((4, 4), dtype=bool))
```

Real-world example of the causal mask: When ChatGPT or Gmail Smart Compose writes the next word, it may only look at words it has *already* written. The mask hides the future, just like covering the answers while you do an exam.

4. Why divide by $\sqrt{d_k}$?

If the numbers in Q and K have mean 0 and variance 1, their dot product has **variance d_k**. Real models use $d_k = 64$, so raw scores have a standard deviation of about 8. Large scores push softmax to an extreme ("saturation"), where one word gets almost 100% of the weight and the gradients become almost zero.

Scores for 3 words	Softmax weights	Gradient size $p(1-p)$ for the top word
Without scaling: [8, 0, −8]	[0.9997, 0.0003, 0.0000]	≈ **0.0003**, so the model almost stops learning
With ÷√64: [1, 0, −1]	[0.665, 0.245, 0.090]	≈ **0.22**, a healthy learning signal

Real-world analogy: a microphone that is too loud. If the volume is too high, every sound clips at maximum and you can't hear any detail. Dividing by $\sqrt{d_k}$ turns the volume down to a level where the differences between words can still be heard.

5. Multi-Head Attention

Instead of one attention calculation, the Transformer runs **h** of them in parallel (8 in the original paper, 12 in BERT-base). Each "head" has its own W_Q, W_K, W_V, so it can learn a different kind of relationship. The results are joined together:

$$\text{MultiHead}(Q,K,V) = \text{Concat}(\text{head}_1, \ldots, \text{head}_h) \cdot W_O$$

Real-world analogy: a team of doctors. A cardiologist, a radiologist and a neurologist examine the same patient and each notices different things. Then they combine their reports. In the same way, one head may track *subject → verb*, another *adjective → noun*, and another *pronoun → the person it refers to*.

6. Positional Encoding

Attention on its own **has no sense of order**: it treats the sentence like a bag of words. Because the Transformer has no recurrence, it adds a position signal made from sine and cosine waves to each word's embedding:

$$PE(pos, 2i) = \sin(pos / 10000^{2i/d}) \quad PE(pos, 2i+1) = \cos(pos / 10000^{2i/d})$$

Khmer example: «ស្តេចទៅបាញ់សត្វ» (*the king goes to shoot the animal*) vs «សត្វទៅបាញ់ស្តេច» (*the animal goes to shoot the king*). Both sentences have exactly the same words. Without positional encoding, the model sees them as identical, even though the meaning is reversed. It's like seat numbers at a concert: the same people in different seats make a different arrangement.

7. The full Transformer architecture

Encoder stack (N layers)

Input embedding + positional encoding
→ Multi-Head **Self**-Attention
→ Feed-Forward Network
(each sub-layer has a residual connection and LayerNorm)
*Job: **understand** the input.*

Decoder stack (N layers)

Masked Multi-Head Self-Attention
→ **Cross**-Attention (looks at the encoder output)
→ Feed-Forward Network
*Job: **generate** the output one token at a time.*

Model family	Uses	Real-world products
Encoder-only (BERT, XLM-R)	Understanding: classification, search, named entities	Google Search ranking, spam filters, sentiment analysis of Facebook comments
Decoder-only (GPT, Claude, LLaMA)	Generating text	ChatGPT, Claude, code assistants
Encoder–Decoder (original Transformer, T5, mBART)	Sequence-to-sequence	Google Translate (Khmer ↔ English), summarization

Common exam trap: BERT is **encoder-only**. It is *not* a full encoder–decoder model.

8. Attention on Khmer text (BERT / XLM-R + BertViz)

- **Subword tokenization:** Khmer doesn't put spaces between words, so a model like `xlm-roberta-base` breaks words into subword pieces. One Khmer word can become 2–4 tokens.
- **Multilingual models:** XLM-R handles Khmer much better than English-only BERT (uncased), because it was trained on 100 languages.

- **BertViz head view:** lets you compare heads, for example which ones link subject → verb.
- **Layer depth:** early layers tend to capture position and neighbouring words, and deeper layers capture meaning (semantics).
- **Open research challenge:** good Khmer word segmentation and tokenization is still an unsolved problem, which makes it a good assignment topic.

9. C5 Critical-Thinking Questions: model answers

Q1. Why use the scaling factor $\sqrt{d_k}$, and how does it prevent vanishing gradients?

The variance of $Q \cdot K^T$ grows in line with d_k. Large scores push softmax into saturation (close to one-hot), where its derivative $p(1-p)$ is close to 0, so almost no gradient flows back and learning stalls. Dividing by $\sqrt{d_k}$ brings the variance back to about 1, so softmax stays in its sensitive range and backpropagation stays stable (see the table in Section 4: a gradient of ≈0.0003 becomes ≈0.22).

Q2. Without positional encoding, what problems would a Transformer face?

- Self-attention is **permutation-invariant**, so any reordering of the words produces the same set of outputs (a "bag of words").
- It can't tell who did what to whom: «ស្តេចទៅជាញ់សត្វ» and «សត្វទៅជាញ់ស្តេច» look the same.
- Negation scope, questions vs statements, and translation word order all break down.
- This is **worse for Khmer**: Khmer has no verb conjugation or case endings, so grammatical roles depend heavily on word order (Subject–Verb–Object).

Q3. Can attention weights be used as evidence of Khmer syntactic structure?

Only as partial, supporting evidence, not as proof. Some heads in BERT do line up with syntax (subject → verb, adjective → noun). But for Khmer:

- Subword splitting means attention links *pieces* of words, not whole words, so the weights have to be merged back into words.
- Many heads send most of their attention to special tokens ([CLS], [SEP], punctuation), which says nothing about syntax.
- Patterns change from head to head and from layer to layer, and attention shows **correlation, not causation**.

To make a stronger claim, compare the patterns against a Khmer dependency treebank and use probing classifiers or ablation (switch a head off and see what breaks).

Q4. Is attention weight alone enough to explain the model's reasoning?

No. Attention shows *where the model looked*, not *why it decided*.

- Information also flows through **residual connections** and **feed-forward layers**, which attention maps don't show.
- Dozens of heads × many layers mix the information together. One head's map is only a small piece of the picture.
- Studies found that very different attention distributions can give the *same* prediction ("Attention is not Explanation", Jain & Wallace, 2019).
- Better tools: **attention rollout/flow**, **gradient-based attribution** (Integrated Gradients), **SHAP/LIME**, **probing** and **ablation**.

> **Analogy:** Eye-tracking can show that a student looked at the right line of an exam question, but that doesn't prove they understood it. Attention is the model's "eye-tracking".

10. Key takeaways

- Attention lets every word look directly at every other word, so there is no bottleneck and the computation runs in parallel.
- **Q** = what I'm looking for, **K** = what I contain, **V** = what I give. Formula: $\text{softmax}(QK^T/\sqrt{d_k})V$.
- $\sqrt{d_k}$ keeps softmax out of saturation, so the gradients stay healthy.
- Multi-head attention works like a team of specialists. Positional encoding gives the model a sense of word order.
- BERT = encoder-only, GPT = decoder-only, and translation = encoder–decoder.
- Attention weights are a useful *clue* for interpretation, but not a full *explanation*.

Based on the lesson "3-5 Attention Mechanism & Transformer Architecture" (Asia Euro University, OBE Level C5). References: Vaswani et al., 2017, "Attention Is All You Need"; Clark et al., 2019, "What Does BERT Look At?"; Jain & Wallace, 2019; Wiegreffe & Pinter, 2019; Abnar & Zuidema, 2020.